

TCP CONCURRENT SERVER

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심

MMcN

멀티미디어 융합 연구실

Mobile Multimedia convergene Network Lab.

<http://mmcn.ajou.ac.kr>



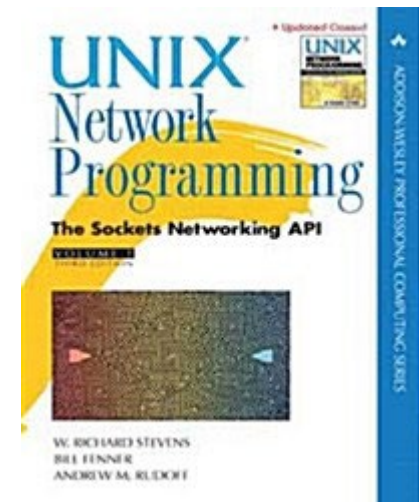
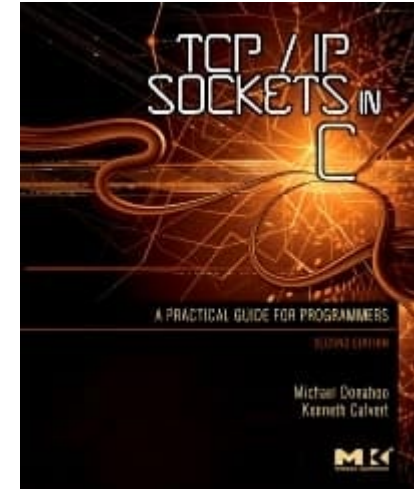
Contents

- ❖ Concept of Concurrent Servers
- ❖ Concurrent Servers using `fork( )`
- ❖ Concurrent Servers using threads
- ❖ Concurrent Servers using `select( )`

References

❖ Main References and Example codes

- TCP/IP Sockets in C, 2nd ed. (2009)
 - authors: Michael J. Donahoo and Kenneth L. Calvert
 - codes for linux
 - <https://jeffdonahoo.github.io/practical/CSockets2/textcode.html>
 - codes for windows
 - <https://jeffdonahoo.github.io/practical/CSockets/winsock.html>
- UNIX Network Programming Volume I-The Sockets Networking, 3rd ed. (2003)
 - authors: W. Richard Stevens, Bill Fenner, and Andrew M. Rudoff
 - codes for linux
 - <http://www.unpbook.com/>

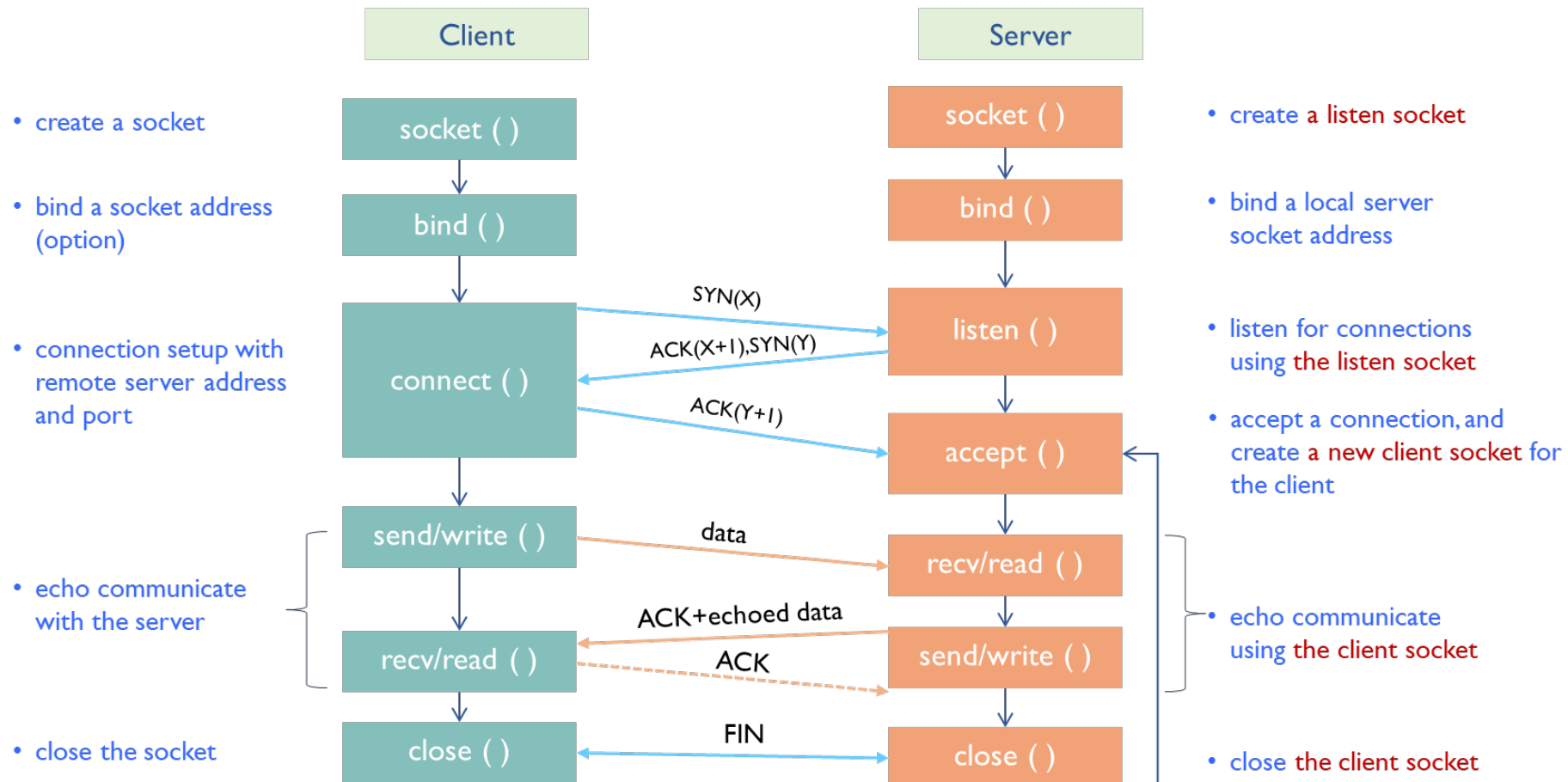




CONCEPT OF CONCURRENT SERVERS

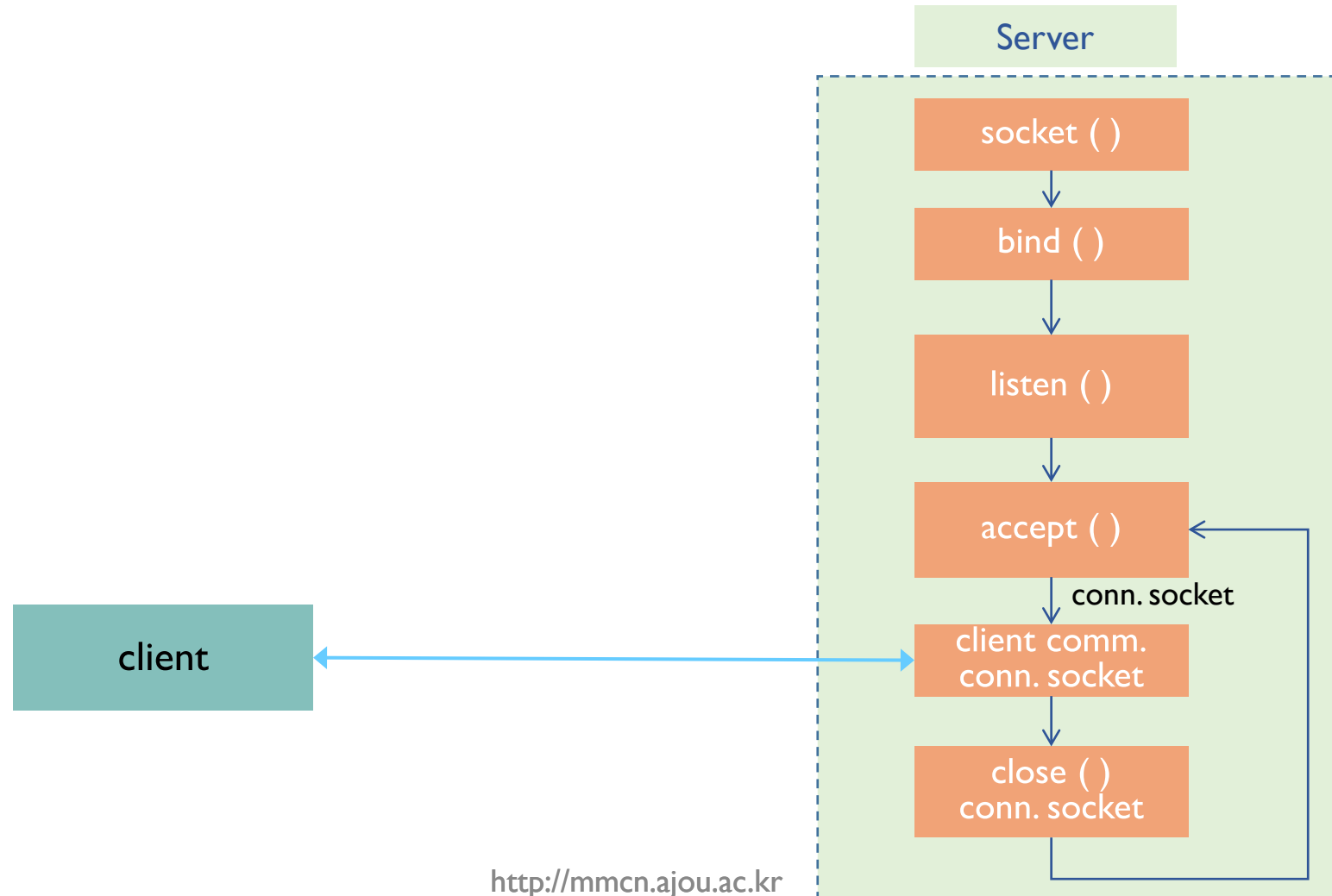
Iterative CS program

❖ TCP echo CS (Client-Server) Program



Iterative CS program

- ❖ Only one client at a time can be served.





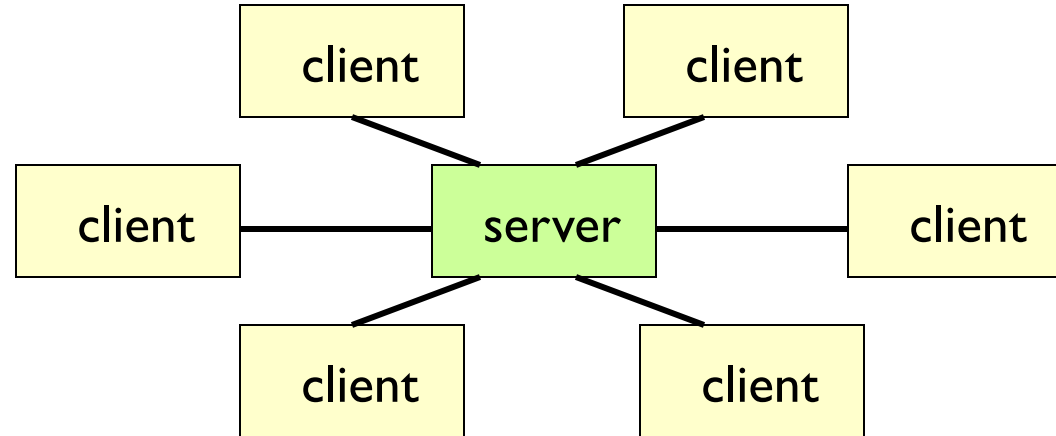
Iterative CS program

- ❖ Iterative (or Sequential) Server
 - Service one request (one client) at a time
- ❖ Why not Iterative (or Sequential) Servers?
 - Increased latency
 - Clients must wait for client I to finish before getting served
 - Low utilization
 - Server is idle while waiting for client's requests.
 - It could have served another client during those idle times!
- ❖ Solution
 - *concurrent servers*
 - Serve multiple clients at the same time

Concurrent Servers

❖ Concurrent Server

- concurrent server with multiple clients



- Basic considerations on server
 - balancing resources among several connected clients
 - dealing with clients as if they have dedicated server access



Concurrent Servers

❖ Possible solutions for concurrent server

■ Multiple **processes**

- Forked server processes: *multi-process*
- Spawn *one server process* to handle each client connection
- Kernel automatically interleaves multiple server processes
- Each server **process** has its *own private address space*

■ Multiple **threads**

- Threaded server processes: *multi-thread* method
- Create *one server thread* to handle each client connection
- Kernel automatically interleaves multiple server threads
- All **threads** share the *same address space*

■ One process using **select()** call: *I/O multiplexing*

■ One process using **epoll()** call: *I/O multiplexing + event*

■ Signal-driven **I/O** (real-time signal)



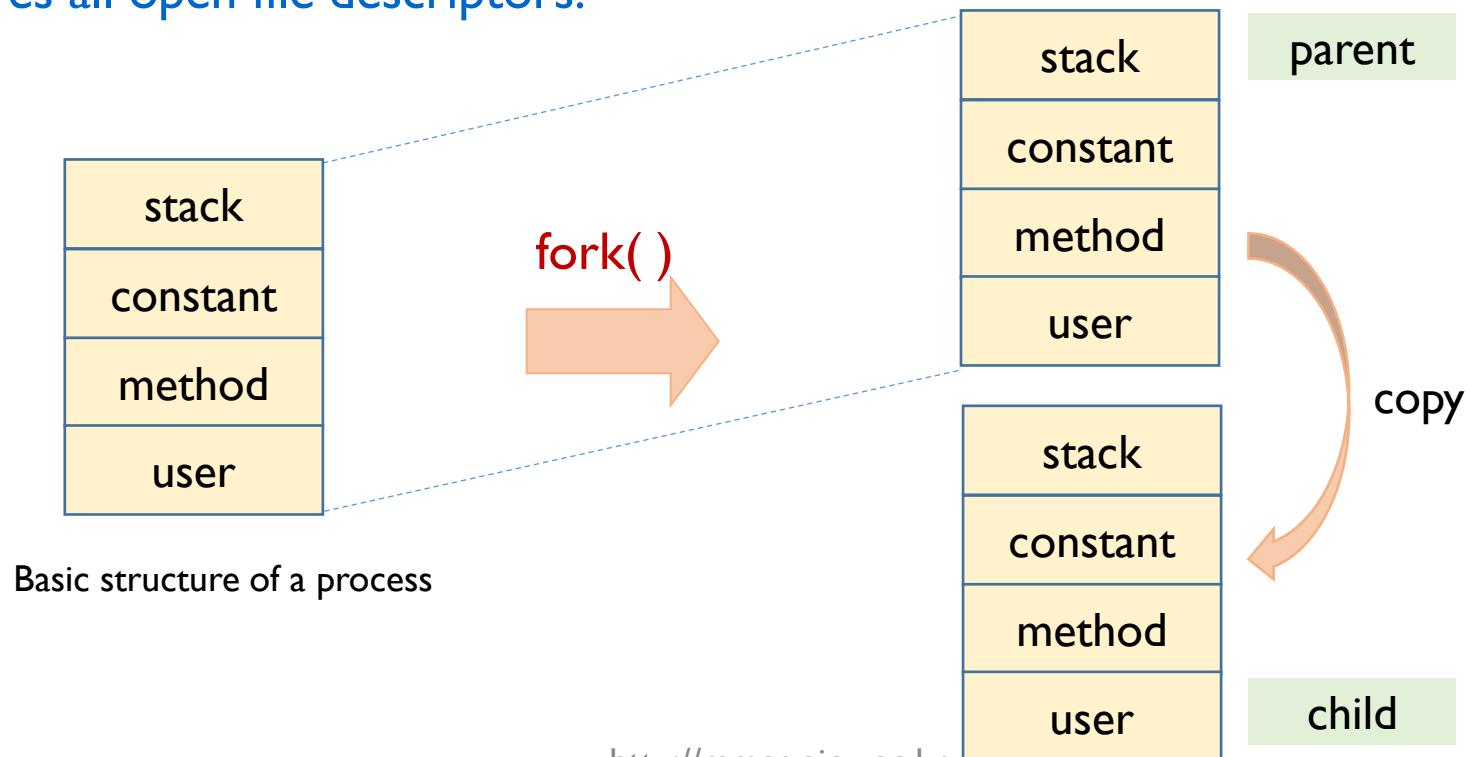
CONCURRENT SERVERS USING FOLK()

Multiple Processes using folk ()

fork()

❖ fork() function

- the only way in UNIX to create a new process
 - it can achieve the way to get process-level parallelism
 - The newly created child process meanwhile has a copy of its parent's environment and shares all open file descriptors.





fork()

❖ fork() syntax

```
#include <unistd.h>

pid_t  fork ( void );
```

■ return values

- in the calling (parent) process
 - process ID of the newly created (the child)
- in the child process
 - 0



fork() example code (1/2)

```
pid_t      pid;
int        listenfd, connfd, retval;

// create socket
listenfd = socket ( PF_INET, SOCK_STREAM, 0 );

// server address/port : omitted
...

// bind socket
retval = bind ( listenfd, ... );

// listening
retval = listen ( listenfd, BACKLOG);
```

fork() example code (2/2)

```
for ( ; ; ) {
```

```
    connfd = accept ( listenfd, ... );
```

return pid=0: the
child process

```
    if ( ( pid = fork( ) ) == 0 ) {
```

```
        close ( listenfd );           // child closes listening socket
```

```
        do_child_process ( connfd ) ; // process the request
```

```
        close ( connfd ) ;           // close the child socket  
        exit ( 0 );                  // terminate the child process
```

```
    }
```

```
    close( connfd );                  // parent closes connected socket  
    ...                               // write a code to clean up all zombies here
```

```
}
```

routine for
the child process

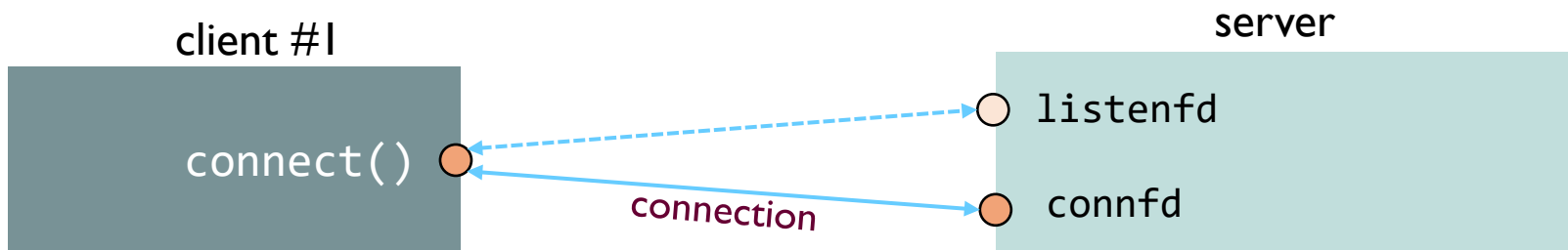
fork() illustration

❖ Status of a client and the concurrent server (1/3)

- before call to `accept( )` returns

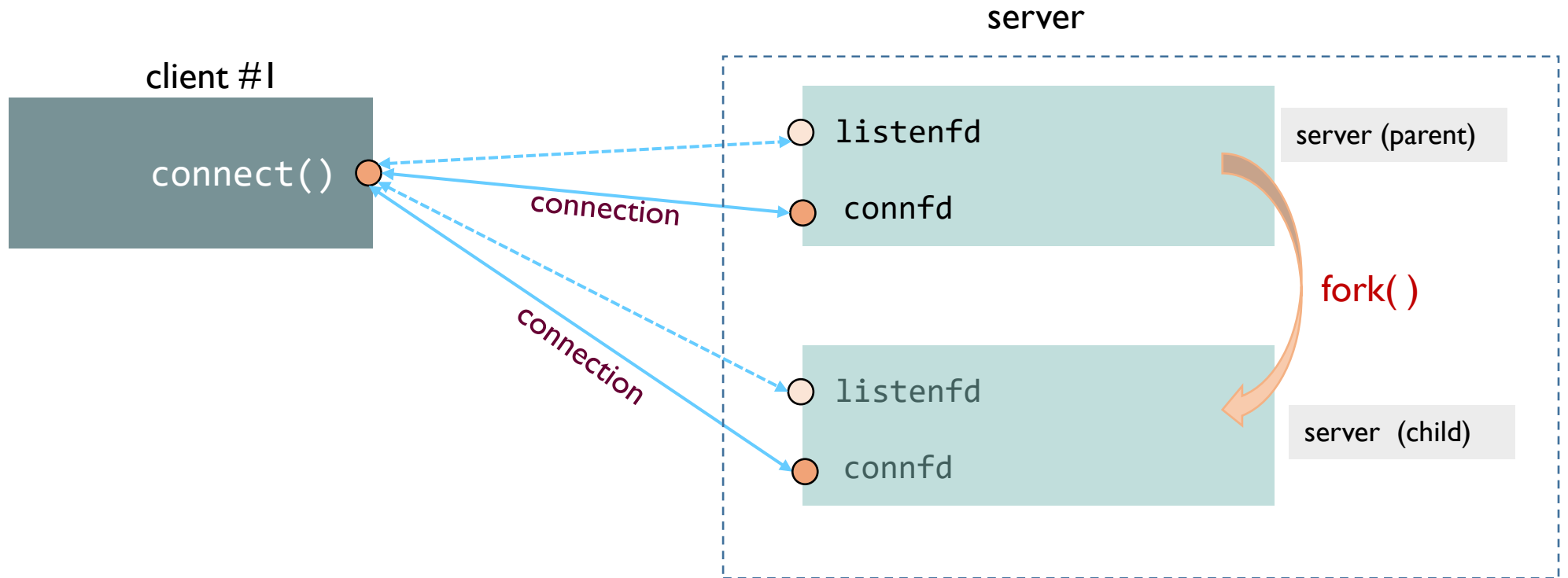


- after return from `accept( )`



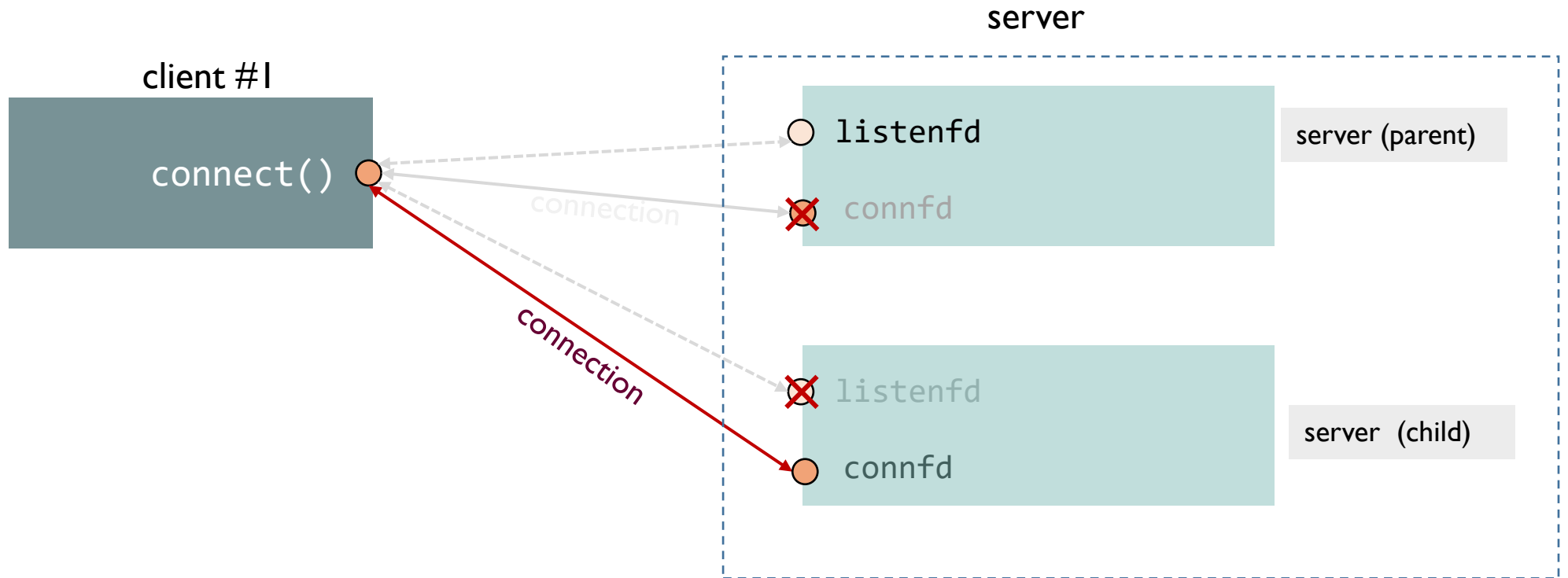
fork() illustration

- ❖ Status of a client and the concurrent server (2/3)
 - after fork() returns



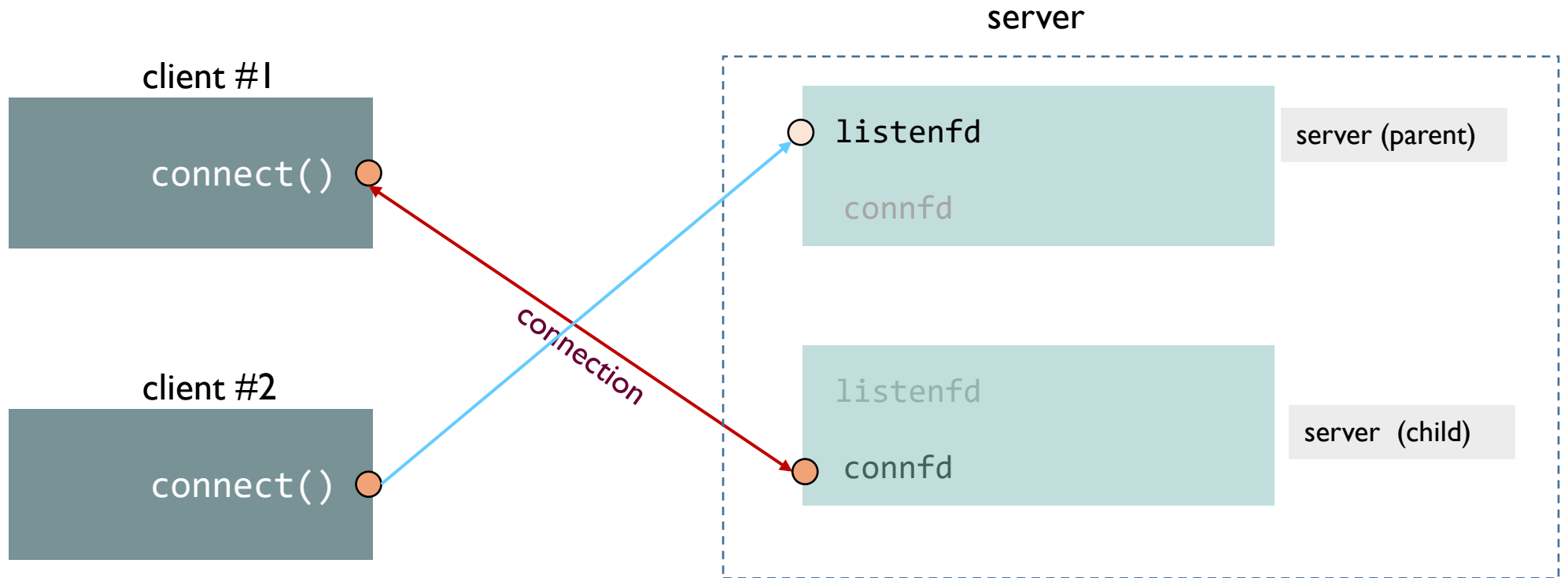
fork() illustration

- ❖ Status of a client and the concurrent server (3/3)
 - after parent and child close appropriate sockets



fork() illustration

- ❖ Status of a client and the concurrent server (3/3)
 - after parent and child close appropriate sockets





Concurrent Server using fork()

- ❖ Using fork() to service multiple clients
 - the simplest way to service multiple-client processes
 - disadvantages
 - larger overhead to copy process
 - difficulties to share information between processes
 - requiring more CPU
 - in Windows OS, fork() is not supported



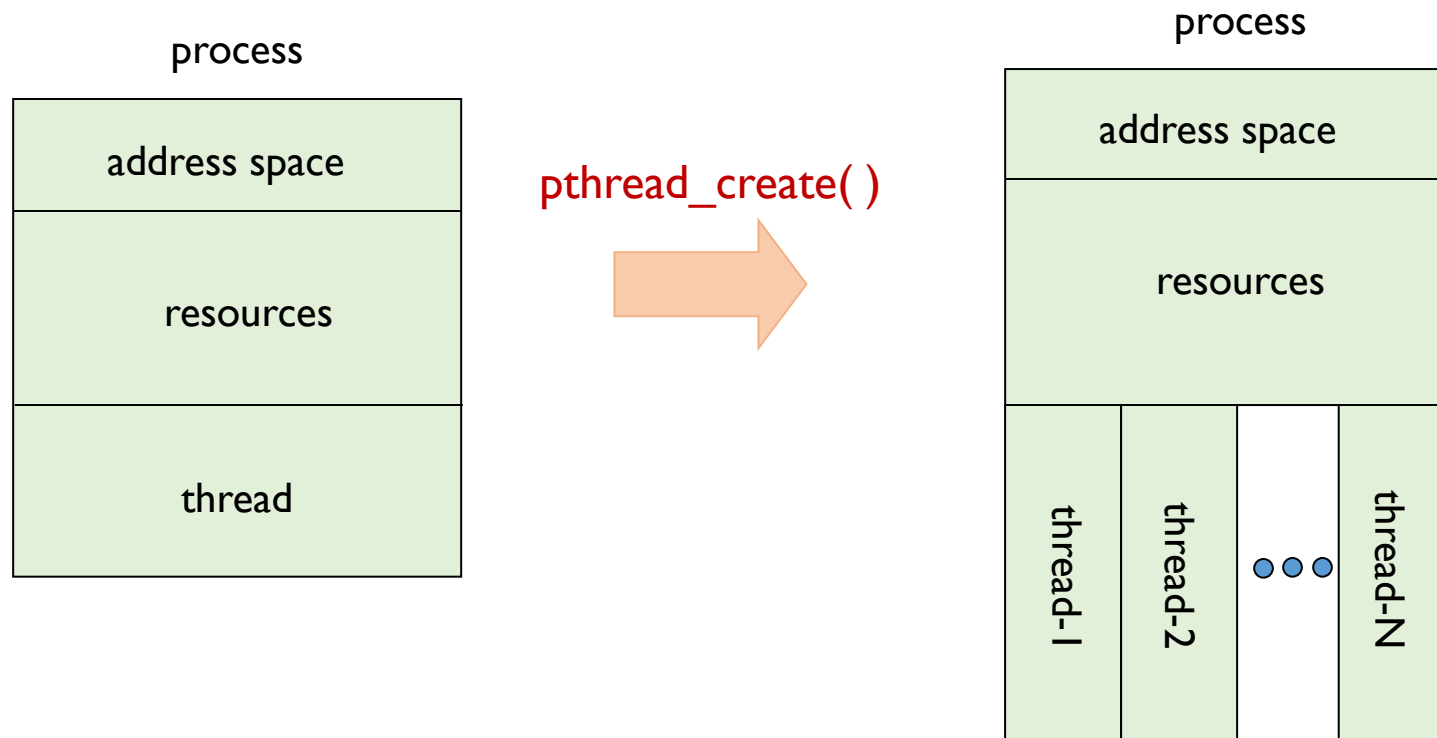
CONCURRENT SERVERS USING THREADS

Multiple Threads using **THREAD**

Thread

❖ Thread

- Threads in a same process share address space, open files, etc.





Thread vs. Process

- ❖ Limitations of the concurrent server using multiple processes
 - Difficult to share information between processes
 - must use IPC (Inter Process Communication)
 - Process creation with fork()
 - relatively expensive
 - time-consuming
- ❖ Advantages of Thread
 - Easy sharing information between threads
 - Copying data into shared (global or heap) variables
 - Must employ the synchronization techniques
 - Faster than process creation
 - typically, ten times faster or better



Thread vs. Process

❖ Disadvantages of Thread

- **Race conditions** - Data protection and Scheduling policy
 - Mutexes and joins must be utilized
- Must consider thread-safe (or thread-safe manner)
- A **bug in one thread** can damage all of the threads in the process
- Competing for use of the finite virtual address space of the host process
 - Large numbers of threads or threads that require large memory??

❖ Check points

- **Signals** in a multithreaded application requires *careful design*
 - As a general principle, it is usually desirable *to avoid the use of signals* in multithreaded programs
- **All threads must be running the same program**, different processes can run different programs
- Aside from data, threads also share certain other information



Thread Functions

❖ pthreads

- a standard set of C library functions (60+ functions) for multithreaded programming
 - IEEE Portable Operating System Interface (POSIX), section 1003.1 standard, 1995
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)

❖ Basic thread functions

- `pthread_create( )` – to create a new thread
- `pthread_self( )` – to return the thread id. of the calling thread
- `pthread_exit( )` – to terminate the calling thread
- `pthread_join( )` – for the calling thread to wait for another thread to terminate
- `pthread_detach( )` – to mark the thread as detached.
- `pthread_kill( )` – to terminate a designated thread

Thread Creation

❖ pthread_create()

```
#include <thread.h>
int pthread_create ( pthread_t      *thread,
                    const pthread_attr_t *attr,
                    void             *(*start_routine)(void*),
                    void             *arg);
```

■ arguments

- **thread** - a buffer to store ID for the thread
- **attr** - Various attributes for the new thread (NULL for default)
- **start** - the main function for the thread
- **arg** - the argument passed to *start*. (i.e., *start(arg)*)

■ return value

- **0** : on success
- **negative values** : on error

Thread Termination

❖ pthread_exit () – terminating the calling thread

```
void      pthread_exit (void *retval);
```

■ arguments

- returns a value via retval that (if the thread is joinable)

❖ pthread_self () - obtaining ID of the calling thread

```
pthread_t  pthread_self ( void );
```

■ return value

- This function always succeeds
- returns a value via retval that (if the thread is joinable)



Joinable and Detached Threads

❖ Thread states

■ Joinable thread (by default)

- the calling thread to wait for another thread to terminate
- when threads terminate
 - resources should be returned to other threads through `pthread_join`
- it is useful in the case that
 - other threads need to know when a thread terminates
 - other threads need to use the return value of a thread function
- the failure to join a joinable thread produces a "zombie thread"
 - each zombie thread consumes some system resources, and
 - when enough zombie threads have accumulated, it will no longer be possible to create new threads (or processes).

■ Detached thread

- no thread can call join on it
- when threads are terminated, their own resources are freed
- it is useful, when threads perform tasks independent of other threads

Joinable and Detached Threads

- ❖ `pthread_detach ( )` – marking the thread as detached

```
int pthread_detach ( pthread_t tid);
```

- arguments

- `tid` – the thread to terminate

- ❖ `pthread_join ( )` - obtaining ID of the calling thread

```
int pthread_join ( pthread_t tid, void **retval );
```

- arguments

- `tid` – the thread id to wait for terminating it
- `retval` – output parameter, value given to `pthread_exit()`

- return value

- `0` on success; an error number on error

Joinable and Detached Threads

❖ Example of joinable threads (1/3)

■ thread function

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

// thread function
void* thread_fn (void *arg)
{
    int id = (int)arg;
    printf ("thread runs (%d)\n", id);
    sleep (id);                      // wait for id second
    id *= 1000;
    printf("terminate thread (%d)\n", id);
    return (void *) (id);
}
```

Joinable and Detached Threads

❖ Example of joinable threads (2/3)

■ main function

```
int main()
{
    pthread_t    t1, t2;
    int          retval;

    pthread_create ( &t1, NULL, thread_fn, (void*)1 );
    pthread_create ( &t2, NULL, thread_fn, (void*)2 );

    pthread_join (t1, (void *)&retval);
    printf("thread join: %d\n", retval);

    pthread_join (t2, (void *)&retval);
    printf("thread join: %d\n", retval);

    return 0;
}
```

- a new thread creates to run
thread_fn with arg 1 (or 2)
- thread's id is t1 (or t2)
- no customized attributes

Wait for thread t1
termination, and retrieve
return value in value_ptr



Joinable and Detached Threads

❖ Example of joinable threads (3/3)

- result

```
thread runs (1)
```

```
thread runs (2)
```

```
terminate thread (1000)
```

```
thread join: 1000
```

```
terminate thread (2000)
```

```
thread join: 2000
```



Thread-based Echo CS Example for Linux (1/3)

```
#include <pthread.h>

void *ThreadMain(void *arg);

struct ThreadArgs {
    int clntSock;
}
```


Thread-based Echo CS Example for Linux (2/3)

```
int main( )
{
    listenfd = socket ( PF_INET, SOCK_STREAM, 0 );    // create a listen socket
    ...                                              // server address/port : omitted
    bind    ( listenfd, ... );                      // bind socket
    listen  ( listenfd, BACKLOG);                   // listening

    for ( ; ; ) {
        cIntSock = accept ( listenfd, ... );

        threadArgs
        threadArgs->cIntSock = cIntSock;

        // create a thread for the accepted client
        pthread_create ( &threadID, NULL, ThreadMain, (void *)threadArgs);
    }
    ...
}
```

Thread-based Echo CS Example for Linux (3/3)

```
void *ThreadMain (void *threadArgs )
{
    int cIntSock;

    // make the thread as detached - its resources are deallocated upon return
    pthread_detach( pthread_self( ) );

    // Extract socket file descriptor from argument
    cIntSock = ((struct ThreadArgs *)threadArgs)->cIntSock;

    free (threadArgs);           // free dynamically allocate memory for argument

    HandleTCPClient ( cIntSock );

    return(NULL);
}
```

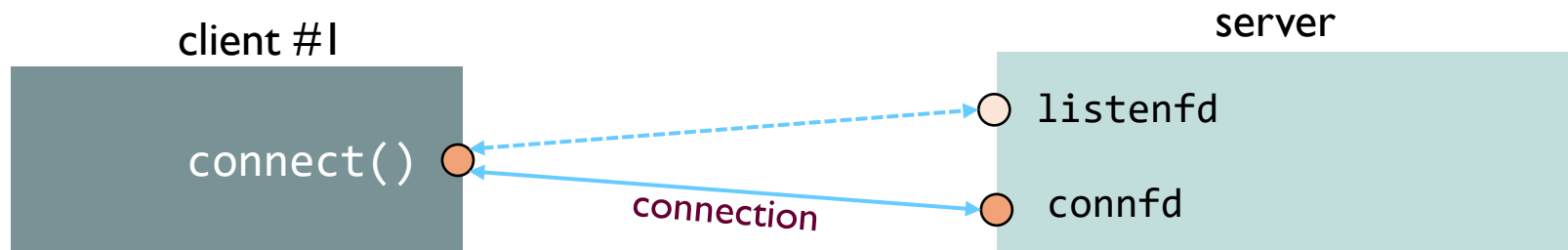
Concurrent Server using thread : illustration (1/4)

❖ Status of a client and the concurrent server

- before call to `accept()` returns

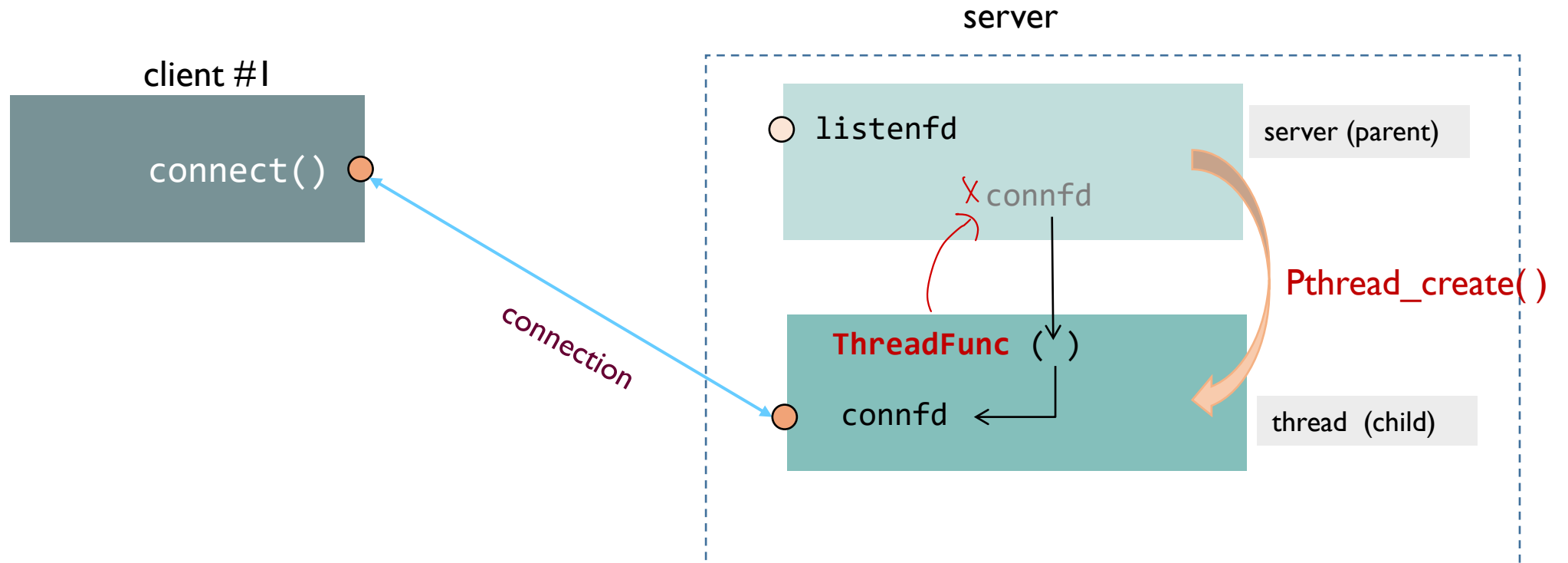


- after return from `accept()`



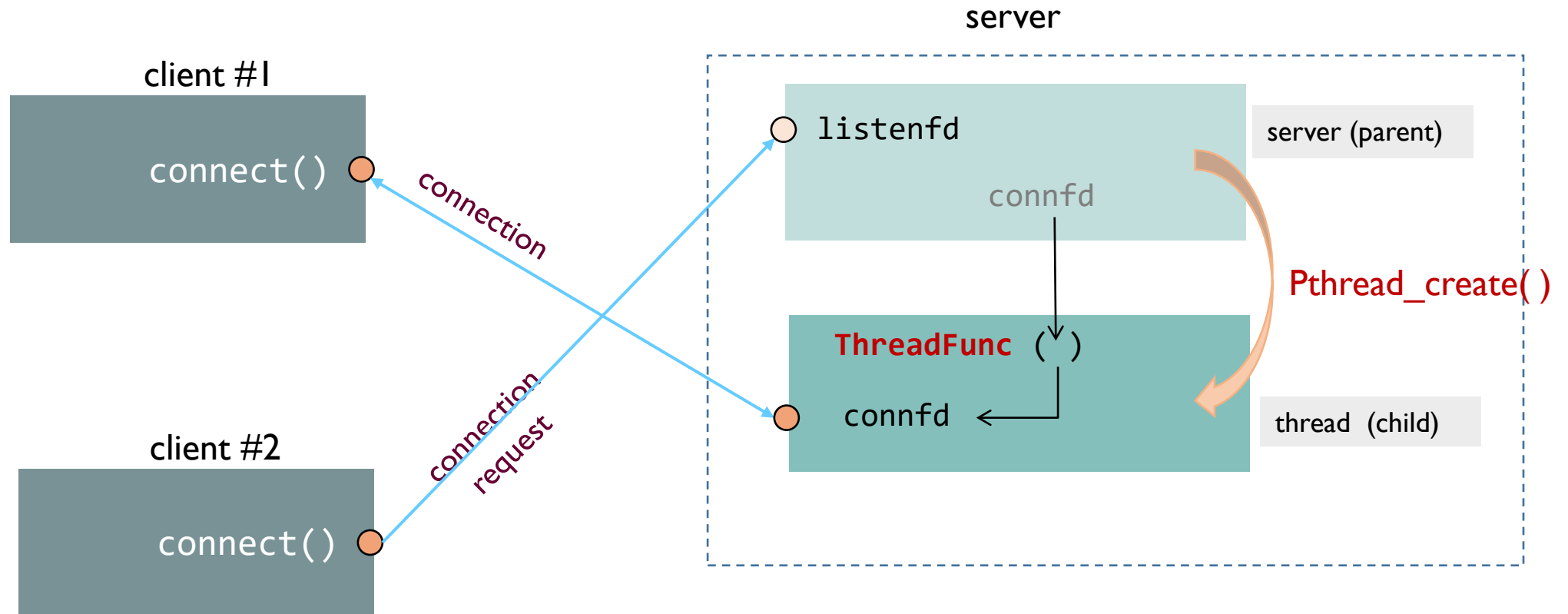
Concurrent Server using thread : illustration (2/4)

- after `pthread_create()`



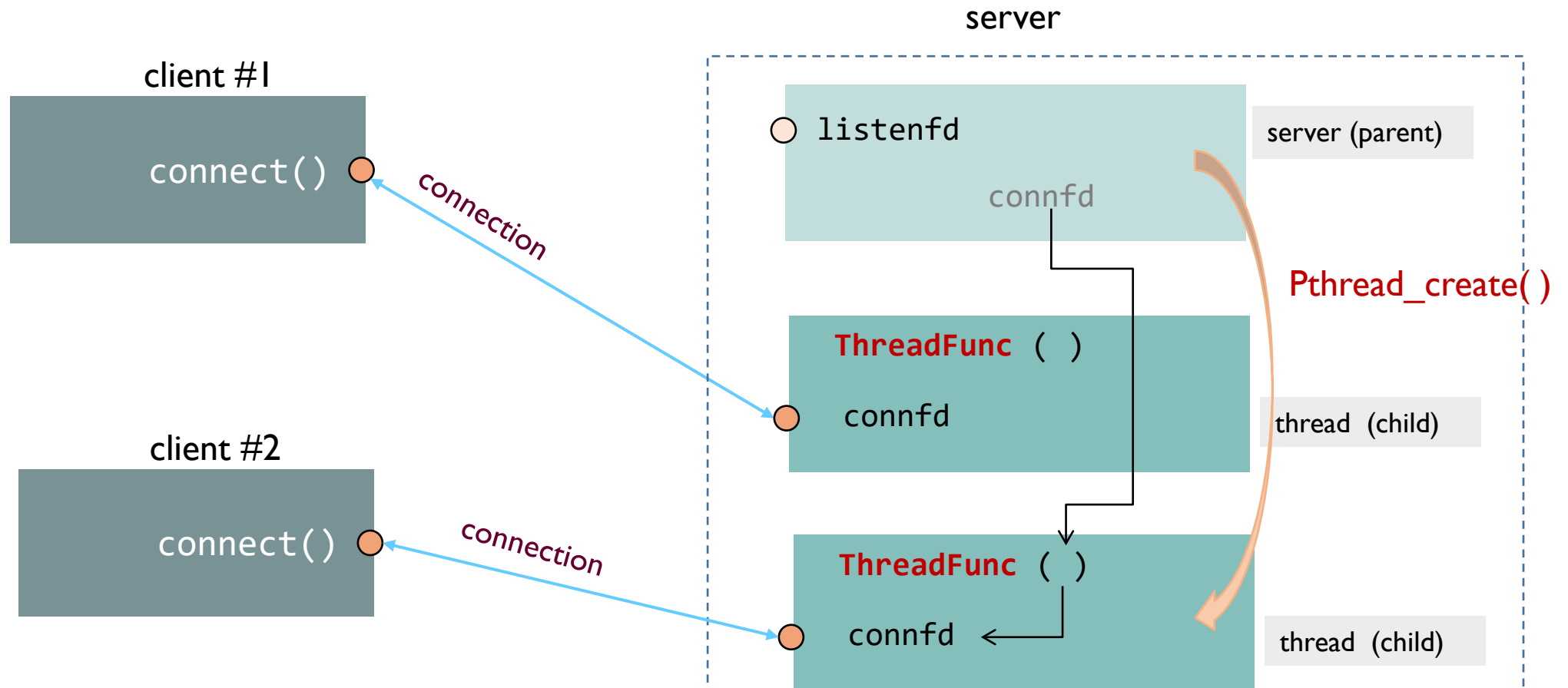
Concurrent Server using thread : illustration (3/4)

- after parent and child close appropriate sockets



Concurrent Server using thread : illustration (4/4)

- after parent and child close appropriate sockets



Use of Thread in Windows OS

❖ Thread Creation in Windows OS

```
HANDLE CreateThread ( LPSECURITY_ATTRIBUTES    lpThreadAttributes,  
                      SIZE_T                    dwStackSize,  
                      LPTHREAD_START_ROUTINE    lpStartAddress,  
                      LPVOID                    lpParameter,  
                      DWORD                     dwCreationFlags,  
                      LPDWORD                   lpThreadId) ;
```

■ arguments

- `lpThreadAttributes` - optional pointer (NULL by default)
- `dwStackSize` - initial size of the stack (0 by default)
- `lpStartAddress` - the main function for the thread
- `lpParameter` - the argument passed to the thread function
- `dwCreationFlags` - flags to control the creation of the thread
- `lpThreadId` - pointer to a variable that receives the thread id.

■ return - thread handle on success, NULL on fail

Use of Thread in Windows OS

❖ Thread Callback function

■ lpStartAddress in **CreateThread ()**

- represents the starting address of an application-defined callback function that serves as the starting address for a thread

■ ThreadProc

- is a placeholder for the application-defined function name.

```
DWORD WINAPI ThreadProc ( LPVOID lpParameter );
```

- example

```
in main:
    HANDLE hThread;
    ...
    hThread = CreateThread (NULL, 0, MyThreadFunction, pData, 0, &dwThreadId);

DWORD WINAPI MyThreadFunction ( LPVOID lpParam )
{ ... }
```


Use of Thread in Windows OS

❖ Terminating a Thread

■ TerminateThread () function

```
BOOL WINAPI TerminateThread (HANDLE hThread, DWORD dwExitCode);
```

- Arguments

- `hThread` : a handle to the thread to be terminate
- `dwExitCode` : The exit code for the thread

- retron

- `nonzero` on success; `zero` on success

■ ExitThread() function

```
VOID WINAPI ExitThread ( DWORD dwExitCode );
```

- Arguments

- `dwExitCode` : The exit code for the thread

- no return value

Thread CS by Windows OS (1/3)

```
int main(int argc, char* argv[])
{
    int                retval, addrlen;
    SOCKET             listen_sock, client_sock;
    SOCKADDR_IN        serveraddr, clientaddr;
    HANDLE             hThread;
    DWORD              ThreadId;

    WSADATA wsa;
    iWSAStartup(MAKEWORD(2,2), &wsa) ;    // initializing winsock

    listen_sock = socket (AF_INET, SOCK_STREAM, 0);

    ZeroMemory(&serveraddr, sizeof(serveraddr));
    serveraddr.sin_family      = AF_INET;
    serveraddr.sin_port        = htons (9000);
    serveraddr.sin_addr.s_addr = htonl (INADDR_ANY);

    retval = bind (listen_sock, (SOCKADDR *)&serveraddr, sizeof(serveraddr));

    retval = listen (listen_sock, SOMAXCONN);
```

Thread CS by Windows OS (2/3)

```
while(1) {
    addrlen = sizeof(clientaddr);
    client_sock = accept (listen_sock, (SOCKADDR *)&clientaddr, &addrlen);

    printf("[TCP Server] connection established to Client (IP: %s, port: %d)\n",
           inet_ntoa(clientaddr.sin_addr), ntohs(clientaddr.sin_port));

    // Creating Thread
    hThread = CreateThread ( NULL, 0, ProcessClient,
                           (LPVOID)client_sock, 0, &ThreadId);

    CloseHandle (hThread);
}

// closesocket()
closesocket (listen_sock);

// 원속 종료
WSACleanup();
return 0;
}
```

Thread CS by Windows OS (3/3)

```
DWORD WINAPI ProcessClient (LPVOID arg ) {
    SOCKADDR_IN clientaddr;
    char        buf[BUFSIZE+1];
    int         addrlen, retval;
    SOCKET      client_sock = (SOCKET)arg;

    addrlen = sizeof(clientaddr);                // obtain client information
    getpeername (client_sock, (SOCKADDR *)&clientaddr, &addrlen);

    while(1) {
        retval = recv (client_sock, buf, BUFSIZE, 0);    // receive data
        buf[retval] = '\\0';                            // display the received data
        printf("Message from client (%s:%d): %s \\n", inet_ntoa (clientaddr.sin_addr),
               ntohs (clientaddr.sin_port), buf);
        retval = send (client_sock, buf, retval, 0);    // echo the data
    }

    closesocket (client_sock);

    ...                                           // displaying some information
    return 0;
}
```

Client 1

```
C:\NetSW\COechoCln.exe
# TCP echo CLIENT
# Network SW Design Course by Ajou University

[Input Data] Client(#1): Hello I am Client #1
[TCP Clnet: send] message 32 bytes sent ...

[TCP Client: rcv] message 32 bytes received.
[Received message] Client(#1): Hello I am Client #1

[Input Data] Client(#1): hello world !!
[TCP Clnet: send] message 26 bytes sent ...

[TCP Client: rcv] message 26 bytes received.
[Received message] Client(#1): hello world !!
```

Client 2

```
C:\NetSW\COechoCln.exe
# TCP echo CLIENT
# Network SW Design Course by Ajou University

[Input Data] 클라이언트(#2): 헬로 나는 클라이언트 #2
[TCP Clnet: send] message 39 bytes sent ...

[TCP Client: rcv] message 39 bytes received.
[Received message] 클라이언트(#2): 헬로 나는 클라이언트 #2

[Input Data] 클라이언트(#2): 하이 헬로 월드 !!
[TCP Clnet: send] message 33 bytes sent ...

[TCP Client: rcv] message 33 bytes received.
[Received message] 클라이언트(#2): 하이 헬로 월드 !!
```

Client 3

```
C:\NetSW\COechoCln.exe
# TCP echo CLIENT
# Network SW Design Course by Ajou University

[Input Data] CLIENT(#3): I AM CLIENT #3
[TCP Clnet: send] message 26 bytes sent ...

[TCP Client: rcv] message 26 bytes received.
[Received message] CLIENT(#3): I AM CLIENT #3

[Input Data] CLIENT(#3): HELLO WORLD .... !!!!
[TCP Clnet: send] message 33 bytes sent ...

[TCP Client: rcv] message 33 bytes received.
[Received message] CLIENT(#3): HELLO WORLD .... !!!!

[Input Data]
```

Server

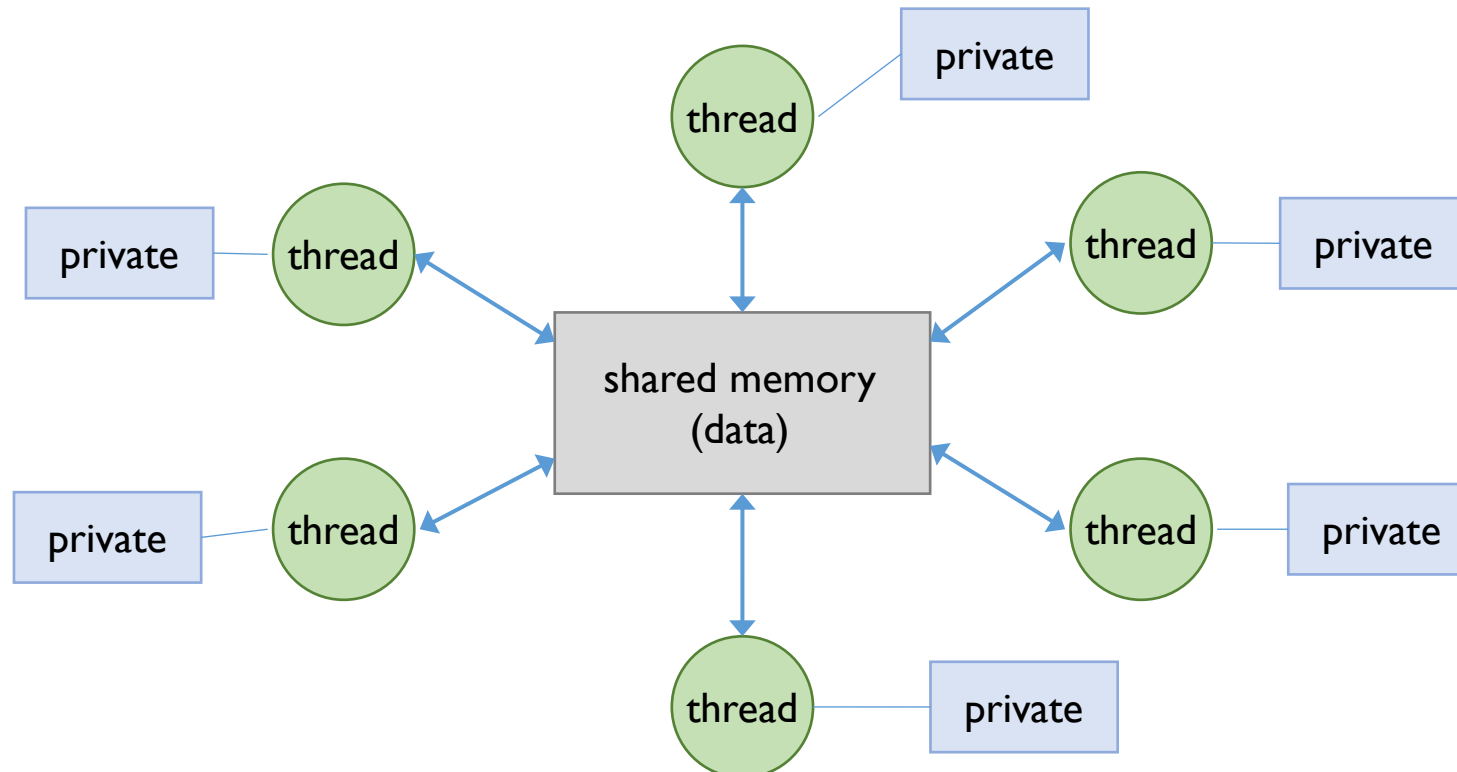
```
C:\NetSW\ThreadSvr.exe
# Concurrent server using multi-thread
# Network SW Design Course by Ajou University

[TCP Server] connection established to Clnet (IP: 127.0.0.1, port: 56660)
[TCP Server] connection established to Clnet (IP: 127.0.0.1, port: 56669)
[TCP Server] connection established to Clnet (IP: 127.0.0.1, port: 56670)
Message from client (127.0.0.1:56660): Client(#1): Hello I am Client #1
Message from client (127.0.0.1:56669): 클라이언트(#2): 헬로 나는 클라이언트 #2
Message from client (127.0.0.1:56670): CLIENT(#3): I AM CLIENT #3
Message from client (127.0.0.1:56660): Client(#1): hello world !!
Message from client (127.0.0.1:56669): 클라이언트(#2): 하이 헬로 월드 !!
Message from client (127.0.0.1:56670): CLIENT(#3): HELLO WORLD .... !!!!
[TCP Server] connection terminated: client (127.0.0.1:56660)
[TCP Server] connection terminated: client (127.0.0.1:56669)
[TCP Server] connection terminated: client (127.0.0.1:56670)
```

Thread Synchronization

❖ Thread Synchronization Problem

- a shared data that threads can read/modify/write
 - multiple threads do not attempt to modify the data at the same time
 - one thread doesn't try to read the data while another thread is modifying it





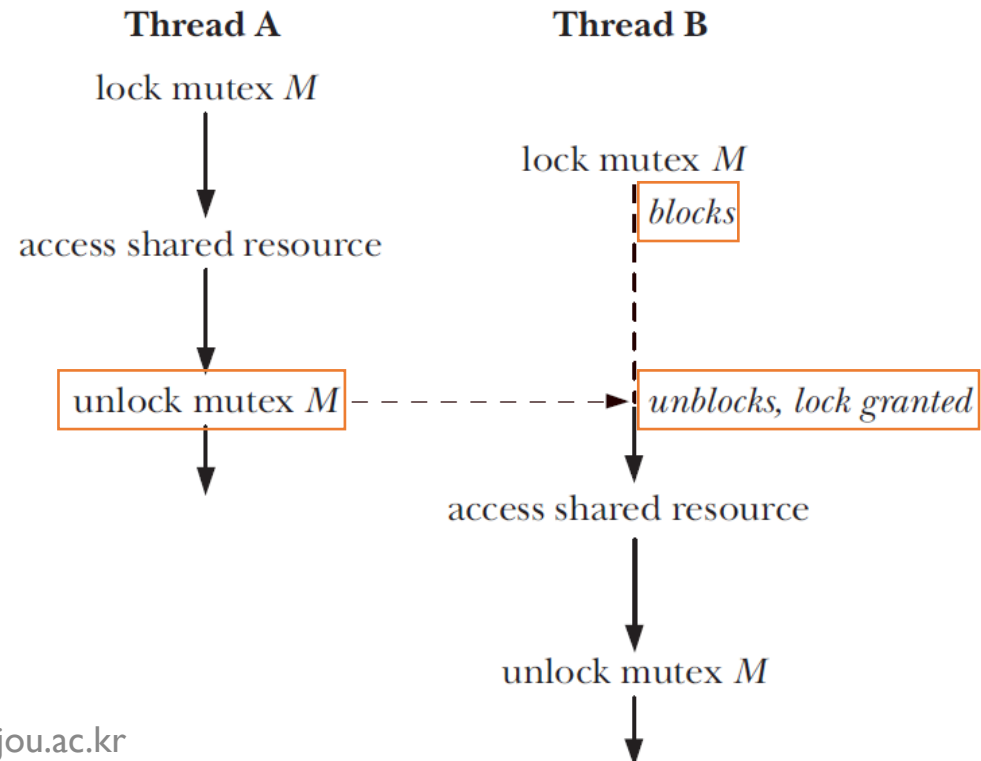
Thread Synchronization

- ❖ High-level synchronization primitives
 - Mutual Exclusion
 - Conditional Variable
 - Semaphore
 - Read-Write Lock

Thread Synchronization

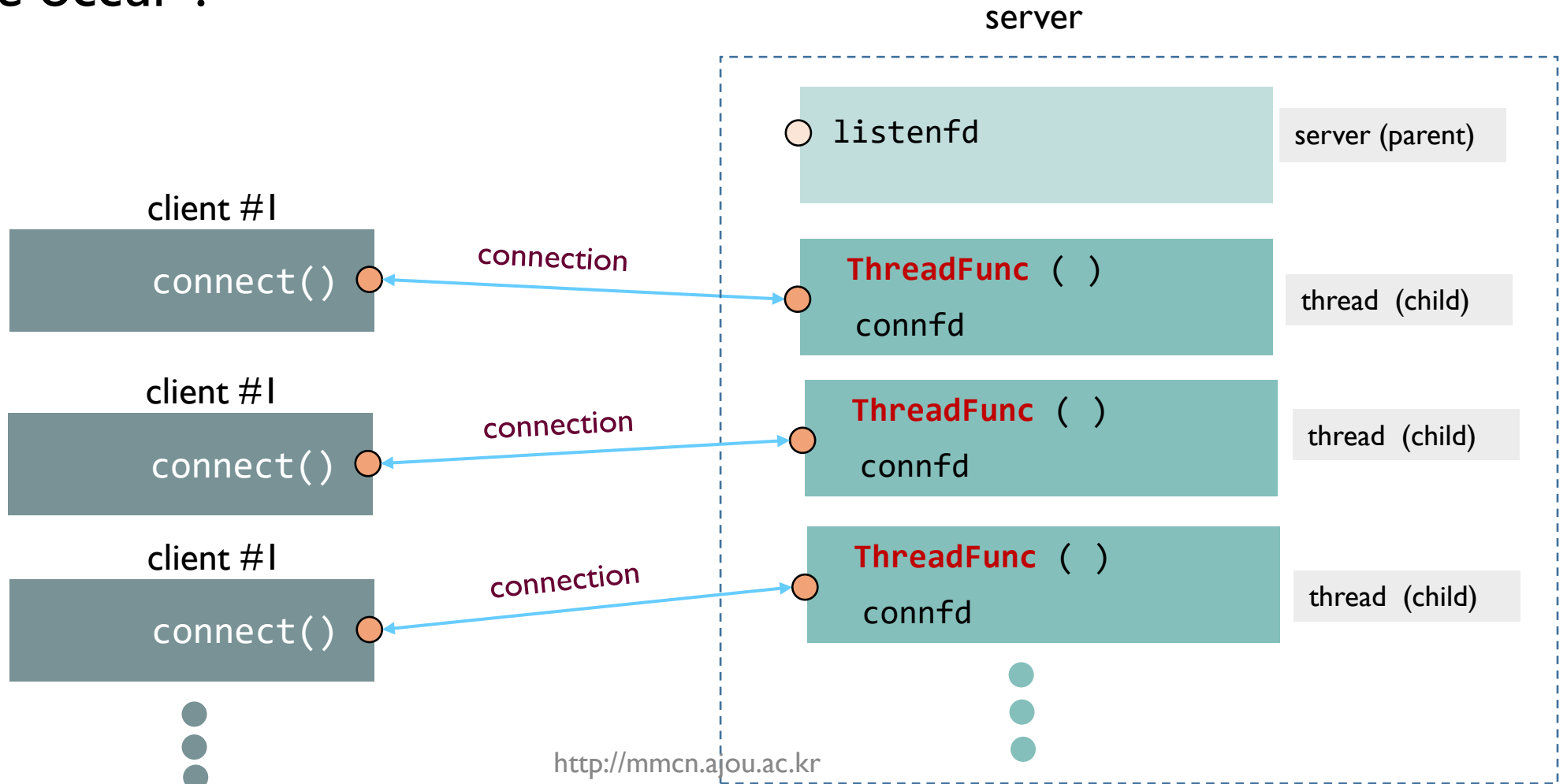
- ❖ Mutual Exclusion (Mutex)
 - Ensure that only one
- ❖ Two states of Mutex – locked & unlocked (acquire and release)
 - At any moment, at most one thread may hold the lock on a mutex

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```



Thread Synchronization

- ❖ In our concurrent-server using thread, does the synchronization issue occur ?

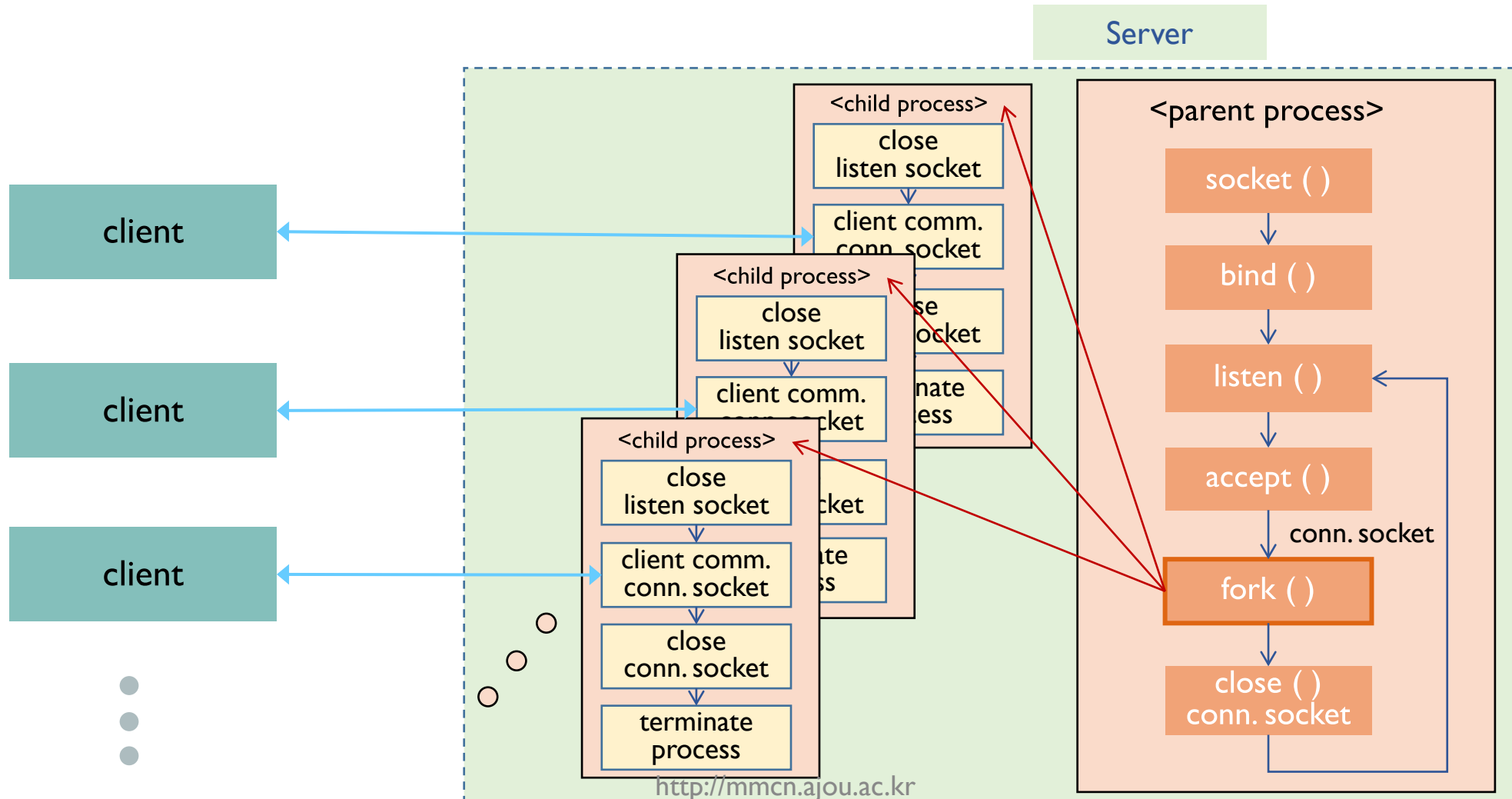


CONCURRENT SERVERS USING SELECT()



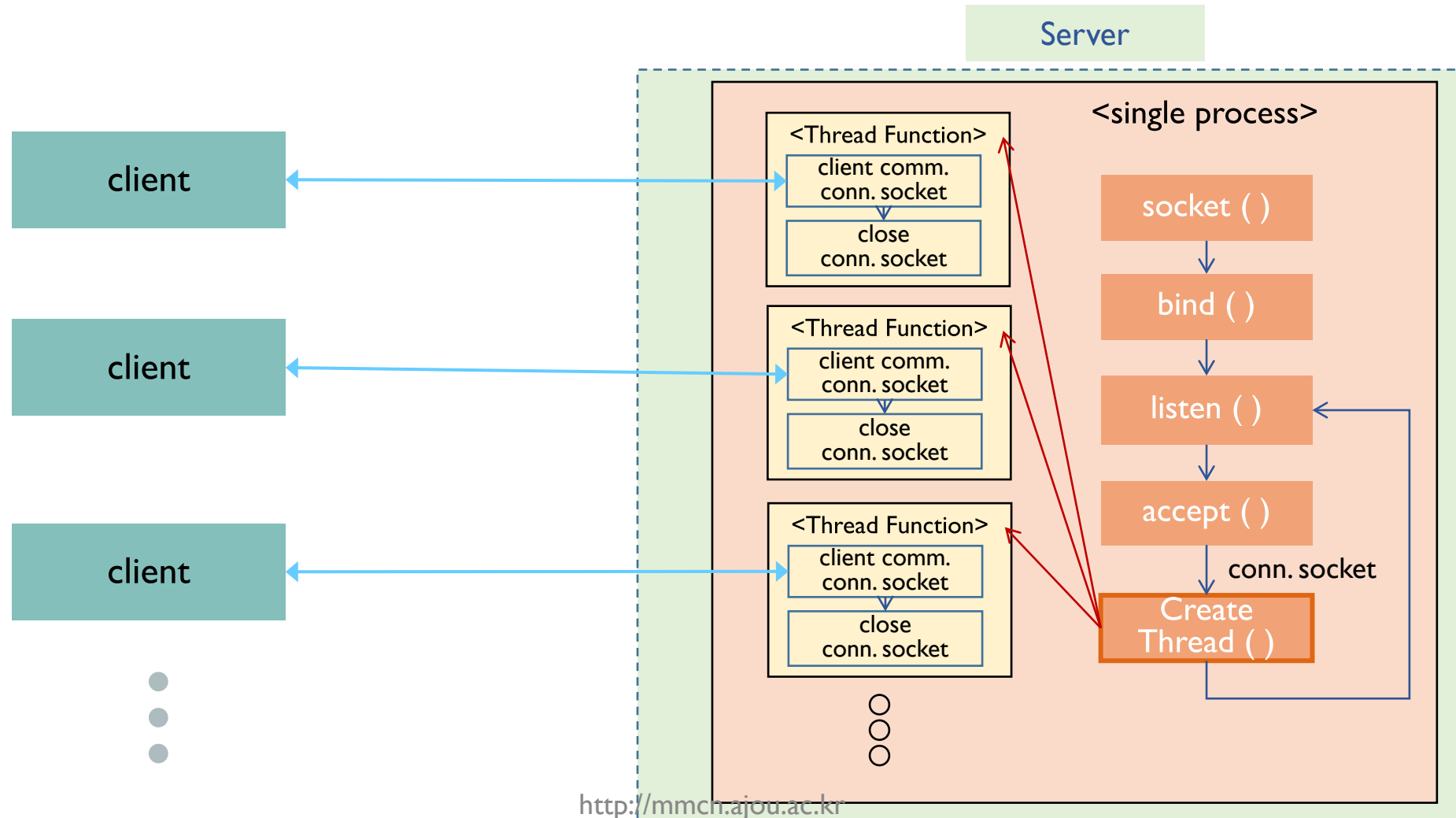
Concurrent Server using fork ()

❖ multiple clients can be served simultaneously.



Concurrent Server using thread

- ❖ multiple clients can be served simultaneously.





Designing Servers Using select()

❖ Considerations on Efficiency

- to share information between connected clients
 - it makes the server to be operated in a **single** process
- to reduce wasting server resources
 - it can be achieved by preventing each process from consuming the same amount of system resources

❖ fork() and thread-based methods

- can achieve the above considerations, but in a very difficult way
 - fork() – IPC should be used for comm. between processes and IPC is not easy to use technique.
 - To synchronize process or thread, skillful technique is required such as IPC or mutex.
- Creating process or thread requires much resources.

select()

❖ select() function

- it blocks the server execution until any of the following events happen:
 - there is something to **read** from a file descriptor
 - there is something to **write** to a file descriptor
 - there is an **exception** on a file descriptor, or
 - specified **time** has been **expired**
- syntax

```
int    select  ( int          n,  
                fd_set        *readfds,  
                fd_set        *writefds,  
                fd_set        *exceptfds,  
                struct timeval *timeout) ;
```

select()

❖ select() function (cont'd)

■ arguments

- **n** : the maximum number of file descriptors (sockets) to test
 - $n = (\text{the highest open socket number}) + 1$
- **readfds** : the set of sockets to be tested for read data 수신
- **writefds** : the set of sockets to be tested for writability 송신
- **exceptfds** : the set of sockets to be tested for exceptions
- **timeout** : the time to wait for one of the specified events to become ready

```
struct timeval {
    long    tv_sec;          /* seconds */
    long    tv_usec;        /* microseconds */
}
```

File Descriptor Set

❖ fd_set

- an array of integers with each bit corresponding to a socket
- for linux

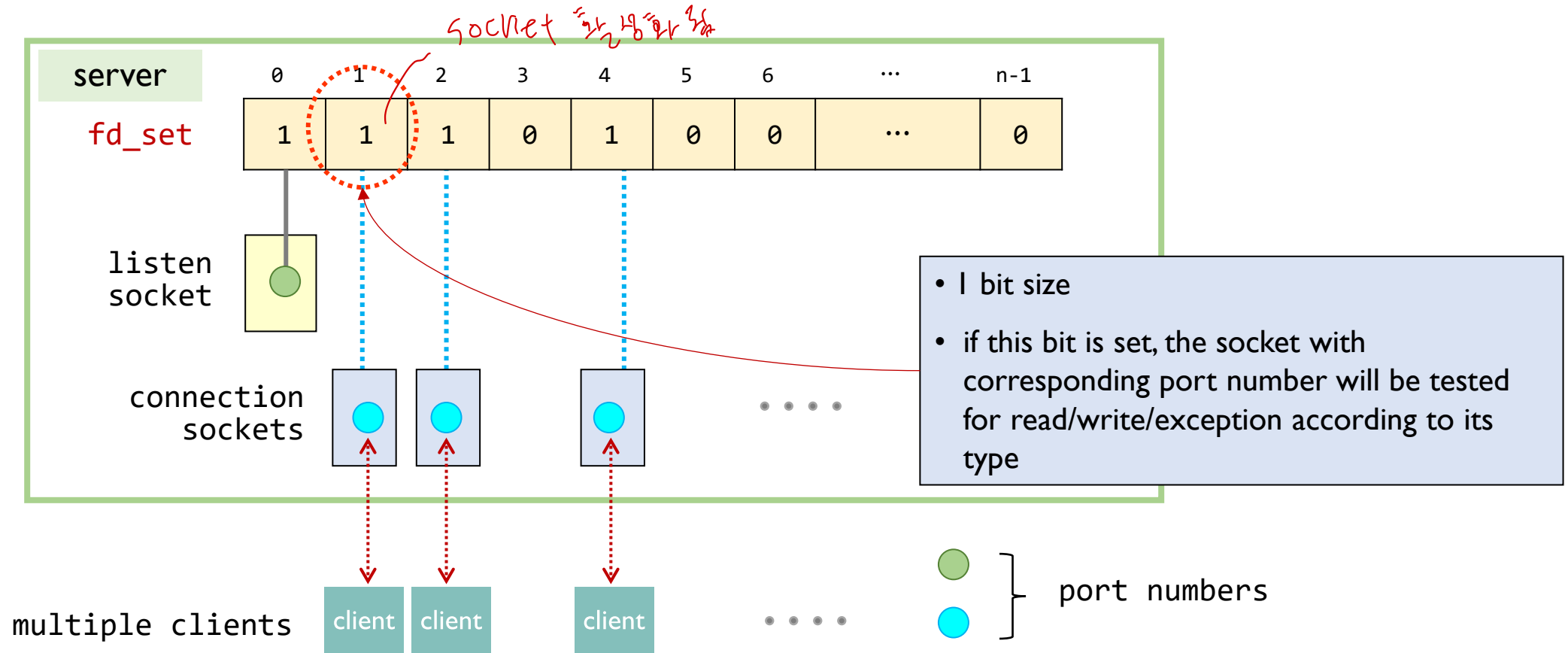
```
//Linux/include/uapi/linux/posix_types.h
#define __FD_SETSIZE      1024
...
typedef struct {
    unsigned long  fds_bits [__FD_SETSIZE / (8 * sizeof(long))];
} fd_set;
```

- for windows

```
typedef struct fd_set {
    u_int      fd_count ;           /* how many are SET? */
    SOCKET     fd_array [FD_SETSIZE]; /* an array of SOCKETS */
} fd_set ;
```


File Descriptor Set

❖ Relations between *fd_set*, *sockets* and *port numbers*



File Descriptor Set

❖ FD_SET Macros

- `void FD_ZERO (fd_set *fdset);`
 - initializes a file descriptor set
 - all bits in `fdset` are cleared (set to 0)
- `void FD_CLR (int fd, fd_set *fdset);`
 - turns on the bit for `fd` in `fdset`
 - set the corresponding bit in `fdset` for the `fd` to 0
- `void FD_SET (int fd, fd_set *fdset);`
 - set the corresponding bit in `fdset` for the `fd` to 1
- `int FD_ISSET (int fd, fd_set *fdset);`
 - tests if the corresponding bit in `fdset` for the `fd` is set to 1
 - returns 1 if it is set to 1,
0 otherwise

File Descriptor Set

❖ FD_SET Macros examples

FD_ZERO(&readfds)



FD_SET(1,&readfds)



FD_SET(2,&readfds)



FD_CLR(1,&readfds)



FD_ISSET(2,&readfds) → 1

0	1	2	3	4	5	6	...	n-1
0	0	0	0	0	0	0	...	0

0	1	2	3	4	5	6	...	n-1
0	1	0	0	0	0	0	...	0

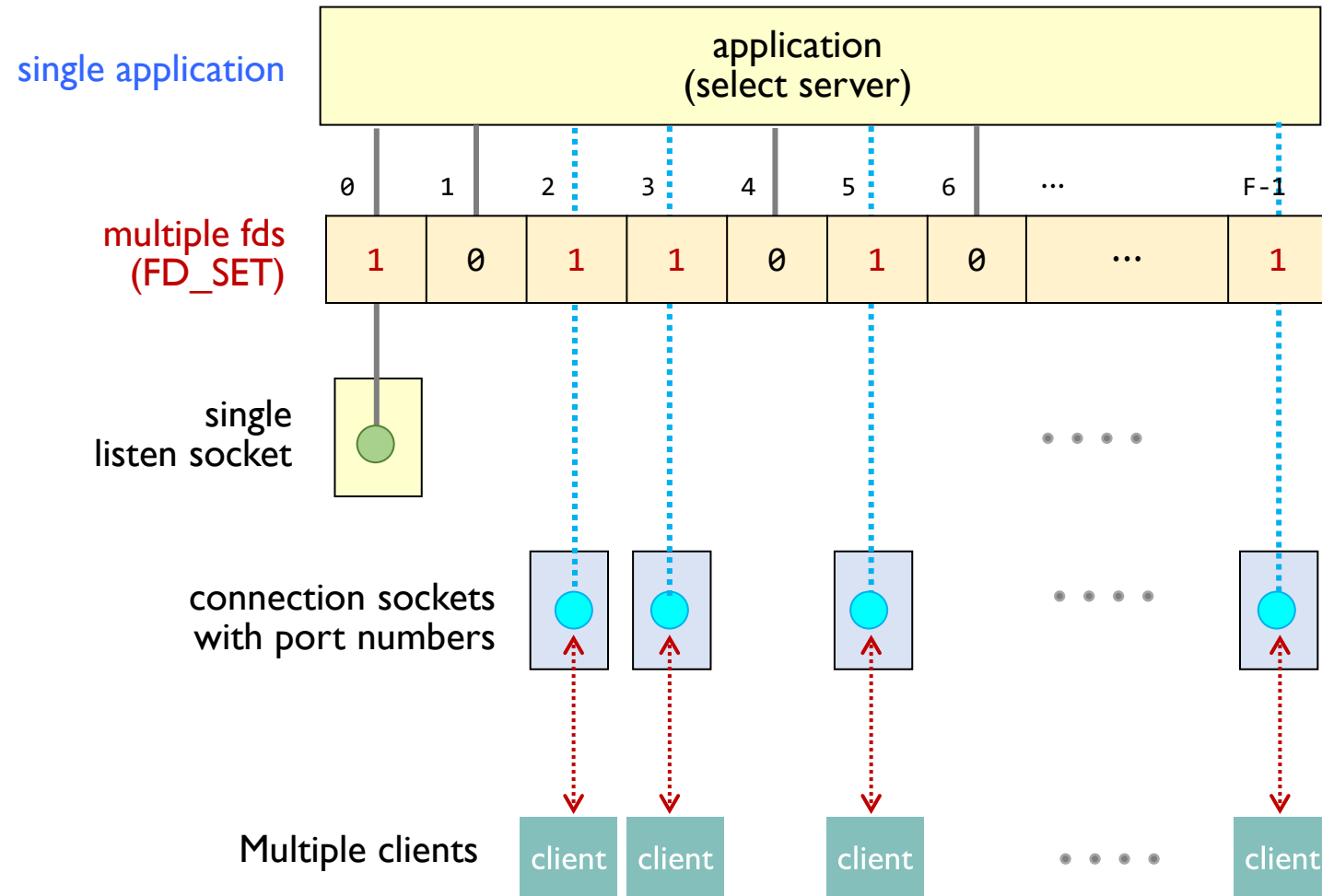
0	1	2	3	4	5	6	...	n-1
0	1	1	0	0	0	0	...	0

0	1	2	3	4	5	6	...	n-1
0	0	1	0	0	0	0	...	0

0	1	2	3	4	5	6	...	n-1
0	0	1	0	0	0	0	...	0

Select Server for Windows Program Example

❖ Overview



Select Server for Windows: Example (1/3)

```
fd_set      readfds, tempfds;
TIMEVAL      timeout; //struct timeval timeout.

// server listen socket creation
svrSock = socket(AF_INET, SOCK_STREAM, 0);

// server address
ZeroMemory(&svrAddr, sizeof(svrAddr));
svrAddr.sin_family      = AF_INET;
svrAddr.sin_addr.s_addr = htonl(INADDR_ANY);
svrAddr.sin_port        = htons(svrPort);

// bind
retval = bind(svrSock, (SOCKADDR*)&svrAddr, sizeof(svrAddr));

// listen
retval = listen(svrSock, SOMAXCONN);

// clear the read fd_sets
FD_ZERO(&readfds);
FD_SET (svrSock, &readfds);    // set for connection attempts (on listen socket)
```

Select Server for Windows: Example (2/3)

```
while(1) {
    tempfds = readfds; // copy readfds to tempfds
    timeout.tv_sec = 5; // timeout value
    timeout.tv_usec = 0;

    // select()
    retval = select( 0, &tempfds, NULL, NULL, &timeout );

    if (retval == 0) continue; // timeout event

    for (i=0; i<readfds.fd_count; i++) {
        if(FD_ISSET(readfds.fd_array[i], &tempfds)) {
            if(readfds.fd_array[i]==svrSock) { // listen event
                // connection from a new client
                clntSock = accept(svrSock, (SOCKADDR*)&clntAddr, &addrlen);

                // add the client's socket to read fd_set
                FD_SET(clntSock, &readfds);
            }
        }
    }
}
```

Select Server for Windows: Example (3/3)

```
else // echo client-server communication with a client
{
    msglen = recv ( readfds.fd_array[i], message, BUFSIZE-1, 0);

    // connection close request from the client
    if(msglen == 0 )
    {
        close (readfds.fd_array[i]);           // socket close

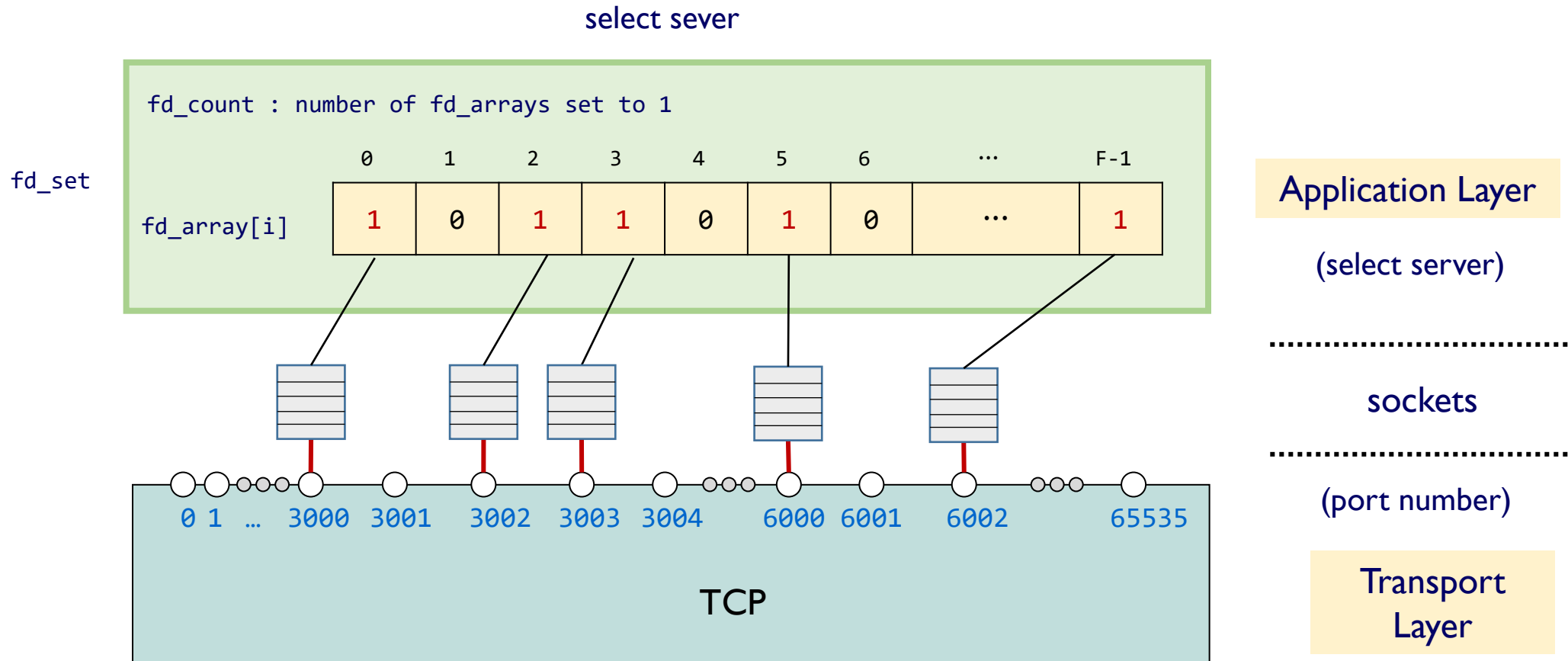
        FD_CLR(readfds.fd_array[i], &readfds); // clear socket from read set
    }
    else
    {
        // send( ): message echo
        send(readfds.fd_array[i], message, msglen, 0);
    }
}

} //if(FD_ISSET())

} //for

} //while
```

FD_SET Illustration for Windows (1/5)



FD_SET Illustration for Windows (2/5)

❖ Initialization and after listen()

■ Initialization

```
FD_ZERO( &readfds )
```

fd_count=0

	0	1	2	3	4	5	6	...	F-1
fd_array[i]	0	0	0	0	0	0	0	...	0

■ after listen()

```
svrSock = socket(AF_INET, ...);  
...  
listen(svrSock, SOMAXCONN);  
...  
FD_SET( svrSock, &readfds );
```

fd_count=1

	0	1	2	3	4	5	6	...	F-1
fd_array[i]	1	0	0	0	0	0	0	...	0

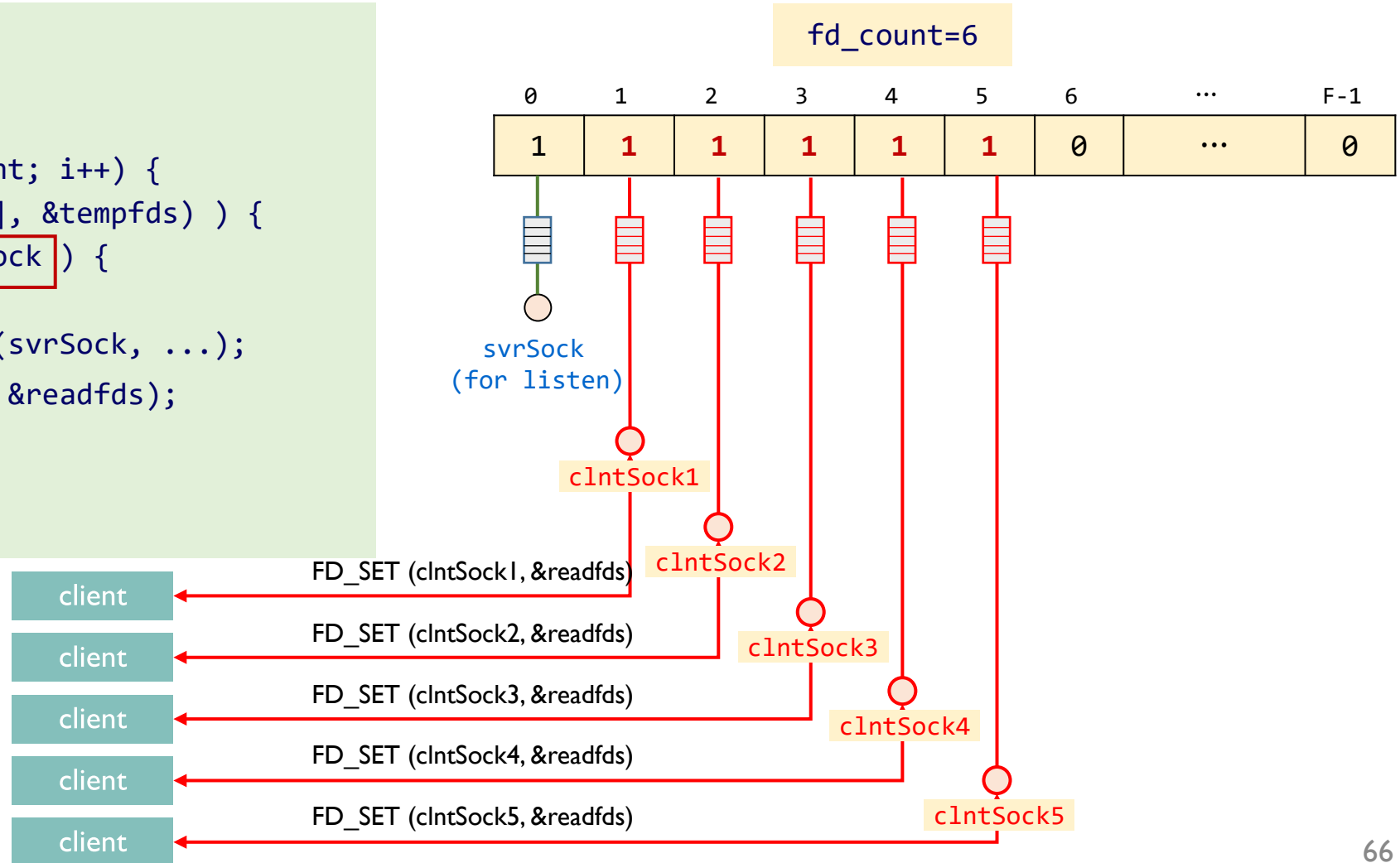


svrSock
(for listen)

FD_SET Illustration for Windows (3/5)

❖ After five clients are connected

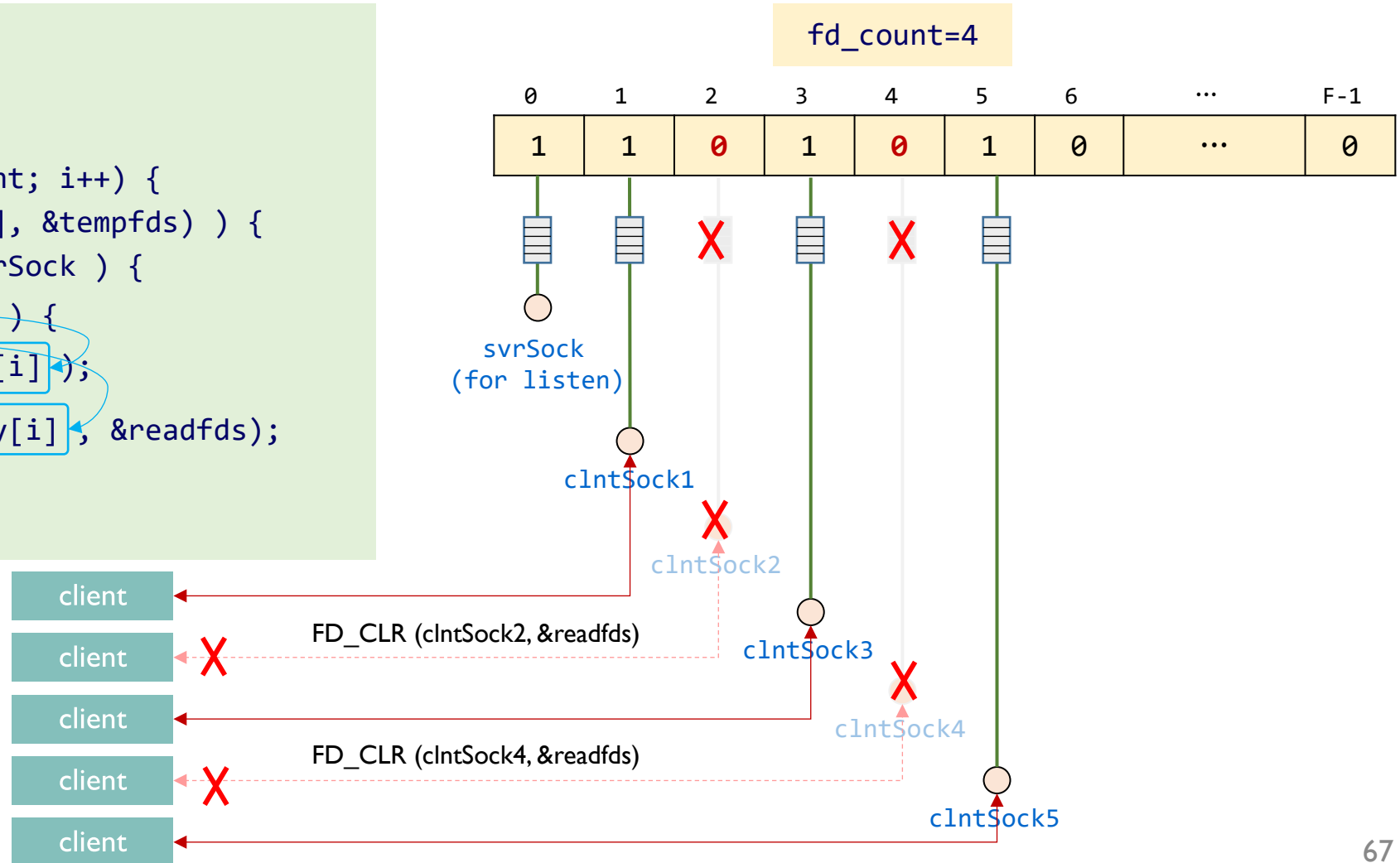
```
while (1) {  
    tempfds = readfds;  
    ...  
    for (i=0; i<readfds.fd_count; i++) {  
        if ( FD_ISSET(fd_array[i], &tempfds) ) {  
            if( fd_array[i]==svrSock ) {  
                ...  
                cIntSock = accept(svrSock, ...);  
                FD_SET( cIntSock, &readfds);  
            }  
        }  
    }  
    ...  
}
```



FD_SET Illustration for Windows (4/5)

❖ After two clients are closed

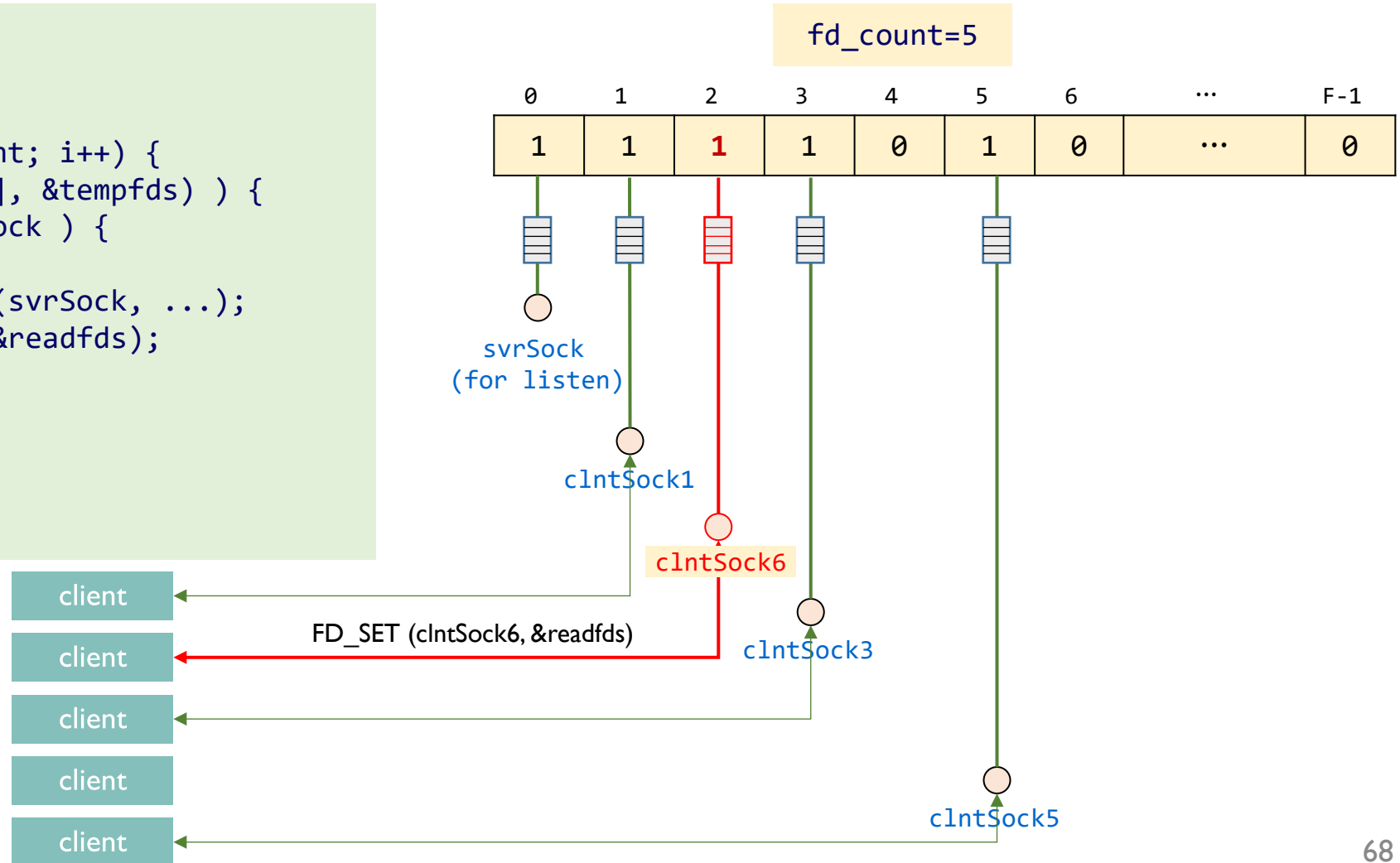
```
while (1) {
    tempfds = readfds;
    ...
    for (i=0; i<readfds.fd_count; i++) {
        if ( FD_ISSET(fd_array[i], &tempfds) ) {
            if( fd_array[i] != svrSock ) {
                if ( close request ) {
                    close( fd_array[i] );
                    FD_CLR( fd_array[i], &readfds);
                }
            }
        }
    }
    ...
}
```



FD_SET Illustration for Windows (5/5)

❖ After a new client is connected

```
while (1) {  
    tempfds = readfds;  
    ...  
    for (i=0; i<readfds.fd_count; i++) {  
        if ( FD_ISSET(fd_array[i], &tempfds) ) {  
            if( fd_array[i]==svrSock ) {  
                ...  
                clntSock = accept(svrSock, ...);  
                FD_SET(clntSock, &readfds);  
            }  
        }  
        ...  
    }  
}
```





Limitations of select()

❖ Servers implemented using select()

- can treat normally up to 1000 ~ 3000 connected clients, at maximum up to 5000

❖ Limitations

- bottleneck using file descriptor set
 - kernel repeatedly checks the entire file descriptor set
 - for higher loads, it causes more CPU loads and longer response time
- the number of clients are limited by FD_SETSIZE
 - default value for Windows: 64

Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

